

Riadenie humanoidého robota NICO

28.1.2026, 15:30-17:00, 19:00-20:30

Andrej Lúčny

<https://github.com/andy1ucny/OAI/workshop>



Ako sa robia takéto systémy?

- **End-to-end systém**

- Natrénuje sa multimodálna neurónová sieť prepájajúca senzory s aktuátormi
- S dnešnými technológiami sa to dá, ale trvá to mesiace a zvláda to len niekoľko subjektov na svete; ostatní nemajú dostatočné technické prostriedky, vedomosti ani rozpočet

- **Modulárny systém**

- Naopak poskladať si modulárny systém môže každý, kto má na to dostatočné vedomosti za niekoľko hodín. **ukážeme ako na to!**

Aké moduly potrebujeme?

Robot kýva hlavou na znamenie áno – nie, takže potrebujeme:

-
-
-
-
-

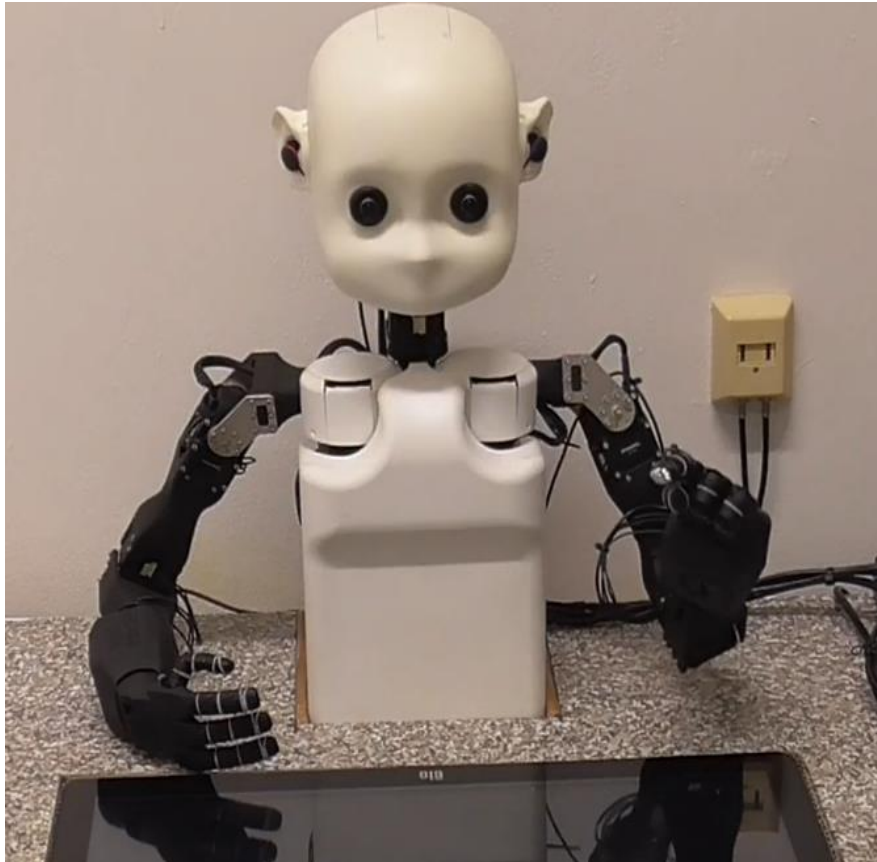
Aké moduly potrebujeme?

Robot kýva hlavou na znamenie áno – nie, takže potrebujeme:

- robota, ktorý má motory v hlave (hardvér alebo simulátor)
- tieto motory sú pripojené na zbernici k počítaču
- kde beží riadiaci softvér
- zídete sa aplikácia, ktorá umožňuje nastavovať jednotlivé DoF a zaznamenávať polohy údov robota
- keď si zistíme aké polohy v akom čase chceme zastávať, potrebujeme knižnicu pomocou ktorej ich z riadiaceho programu nastavujeme

Čo potrebujeme na implementáciu?

1

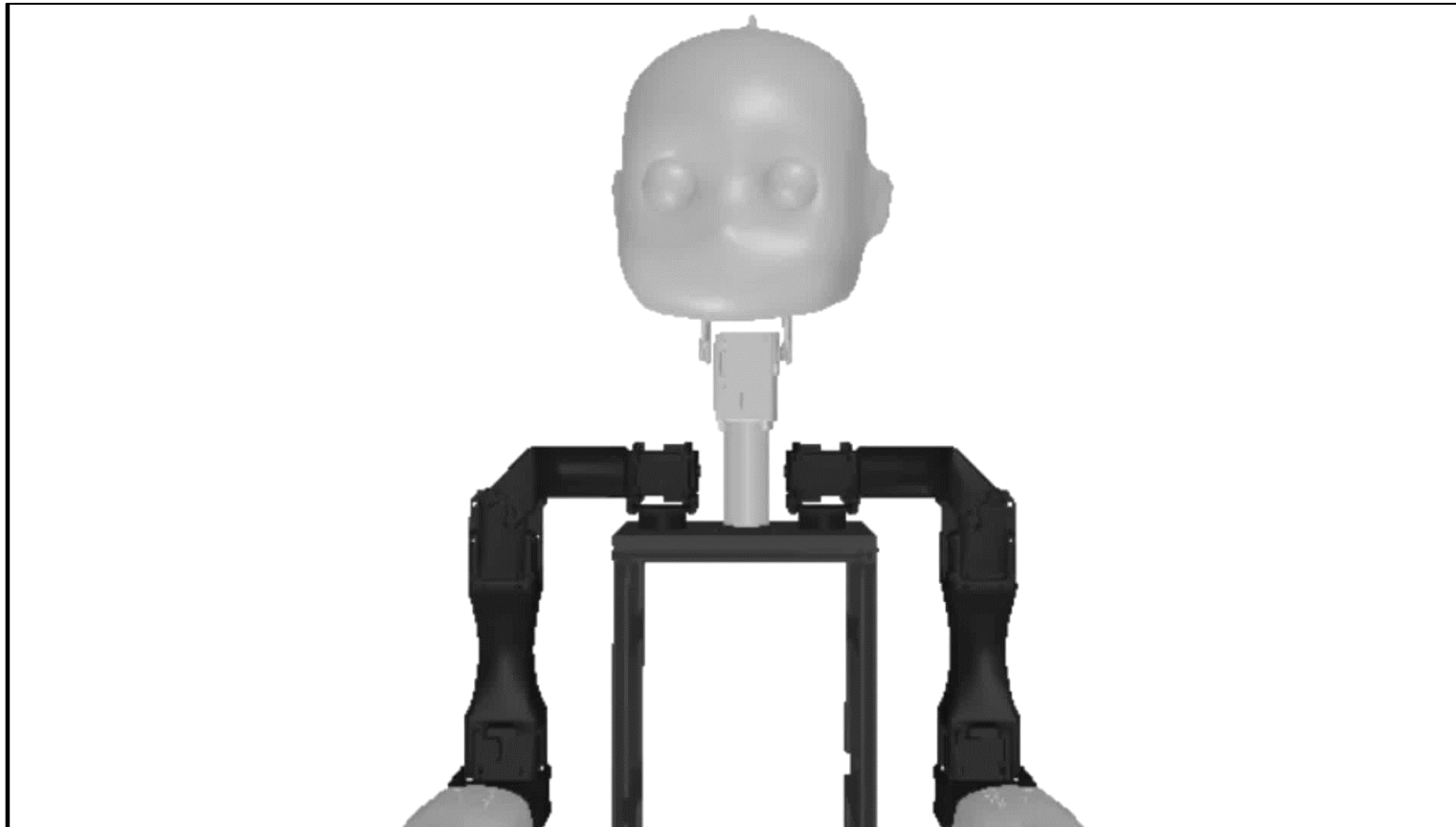


- Robot NICO (Neuro-Inspired Companion), vyvinutý na Univerzite v Hamburgu
- 6 + 4 DoF na každej ruke
seed robotics EROS ax-12w
servomotory s encoderom
- 2 senzory úchytu v dlani
- 2 DoF v krku
- 2 kamery See3CAM CU135
- reproduktor
- 2 mikrofóny
- LED na zobrazenie emócie



Čo potrebujeme na implementáciu?

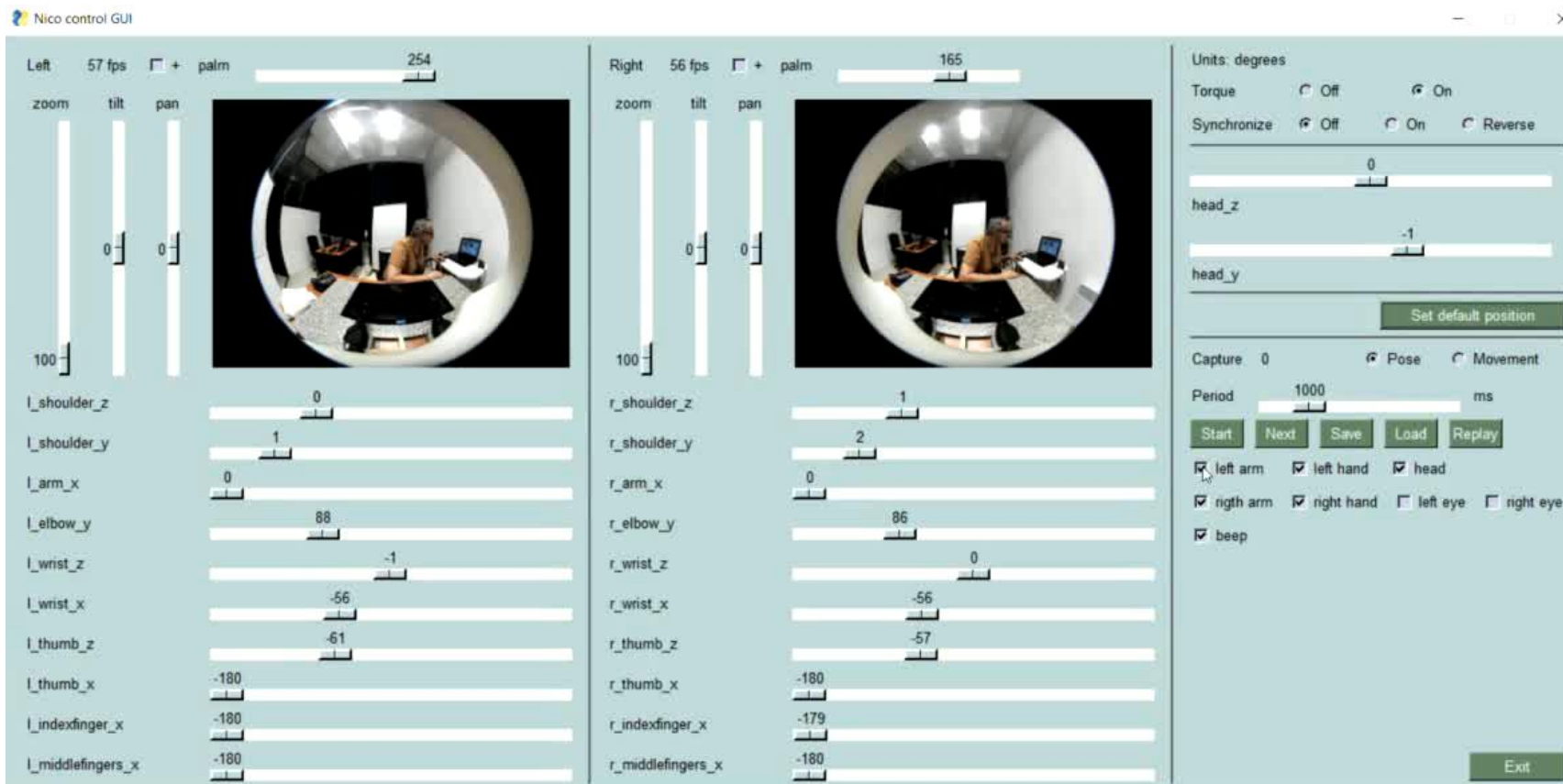
1



- Alternatívne môžeme použiť simulátor a kameru (stačí web kamera z notebooku)

Čo potrebujeme na implementáciu?

1



- Vyvinuli sme aplikáciu NicoGUI, v ktorej ľahko zistíme potrebné hodnoty uhlov a ich správne načasovanie na hýbanie hlavou robota

Čo potrebujeme na implementáciu?

2

```
movement = [ # Ano
    {'head_y':-20},
    {'head_y':20},
    {'head_y':-20},
    {'head_y':20},
    {'head_y':-20},
    {'head_y':20},
]
#
durations = [0.5,0.5,0.5,0.5,0.5,0.5]
play_movement(movement,durations)
```

```
movement = [ # Nie
    {'head_z':-30},
    {'head_z':30},
    {'head_z':-30},
    {'head_z':30},
    {'head_z':-30},
    {'head_z':30},
]
```

- Na prehratie pohybu hlavy robota používame vlastnú knižnicu nicomover opierajúcu sa o knižnice nicomotion, pypot a Dynamixel SDK

Aké moduly potrebujeme?

Robot vidí, čo mu ukazujeme, takže potrebujeme:

-
-



TERAIS



Co-funded by
the European Union

Aké moduly potrebujeme?

Robot vidí, čo mu ukazujeme, takže potrebujeme:

- kameru
- OpenCV



Čo potrebujeme na implementáciu?

4

- Prostriedok integrácie
- Bez neho je programovanie už len takého jednoduchého systému tak náročné, že sa to prakticky nedá zvládnuť
- Systém beží v reálnom čase

Integrácia

- Použijeme architektúru čiernej tabule (rozšírené dictionary)

```
from agentspace import space
```

```
space['a'] = 2
```

```
print(space['a']) # 2
```

```
print(space['b']) # None
```

```
print(space(default=-1)['b']) # -1
```

Integrácia

```
space(validity=0.5)['b'] = 1
print(space['b']) # 1
time.sleep(1)
print(space(default=-1)['b']) # -1
```

```
space['c'] = 1 # default priority is 1.0
space(validity=0.5,priority=2.0)['c'] = 2
space(validity=2.0,priority=1.5)['c'] = 3
print(space(default=0)['c']) # 2
time.sleep(1)
print(space(default=0)['c']) # 0
```

```
from agentspace import Agent

class MyAgent(Agent):

    def __init__(self, args):
        process(args)
        super().__init__()

    def init(self):
        self.attach_timer(1.0)

    def senseSelectAct(self):
        print('hallo')
```

MyAgent()

agents (objects that
have own thread)

invoked by timer

Integrácia


```
from agentspace import Agent
```

agents (objects that
have own thread)

```
class MyAgent(Agent):
```

```
    def __init__(self, args):  
        process(args)  
        super().__init__()
```

invoked by trigger

```
    def init(self):  
        space.attach_trigger('a', self)
```

```
    def senseSelectAct(self):  
        print('hallo')
```

```
MyAgent()
```

Integrácia

Aké moduly potrebujeme?

5

Robot sa otáča za ukázanými (akýmikoľvek) predmetmi, takže potrebujeme:

-
-
-

Aké moduly potrebujeme?

Robot sa otáča za ukázanými (akýmikoľvek) predmetmi, takže potrebujeme:

- potrebujeme spracovať obraz, tak aby sme na ňom vedeli nájsť, kde je blízky predmet
- potrebujeme otáčať hlavou, aby sme predmet mali v strede obrazu



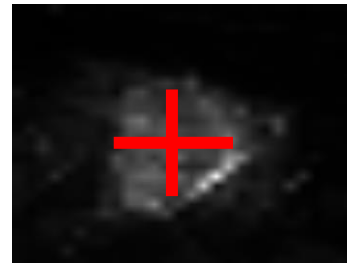
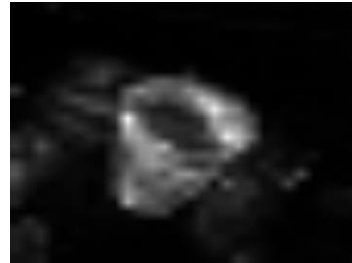
TERAIS



Co-funded by
the European Union

Čo potrebujeme na implementáciu?

5

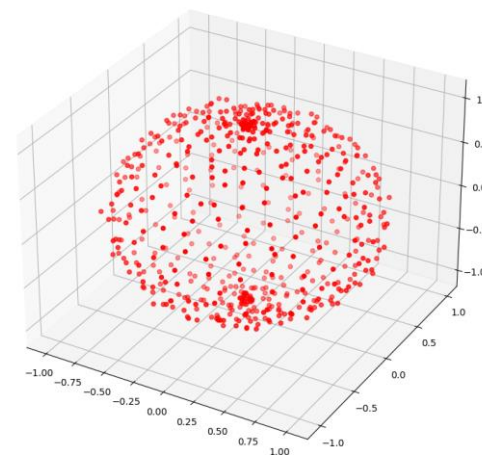
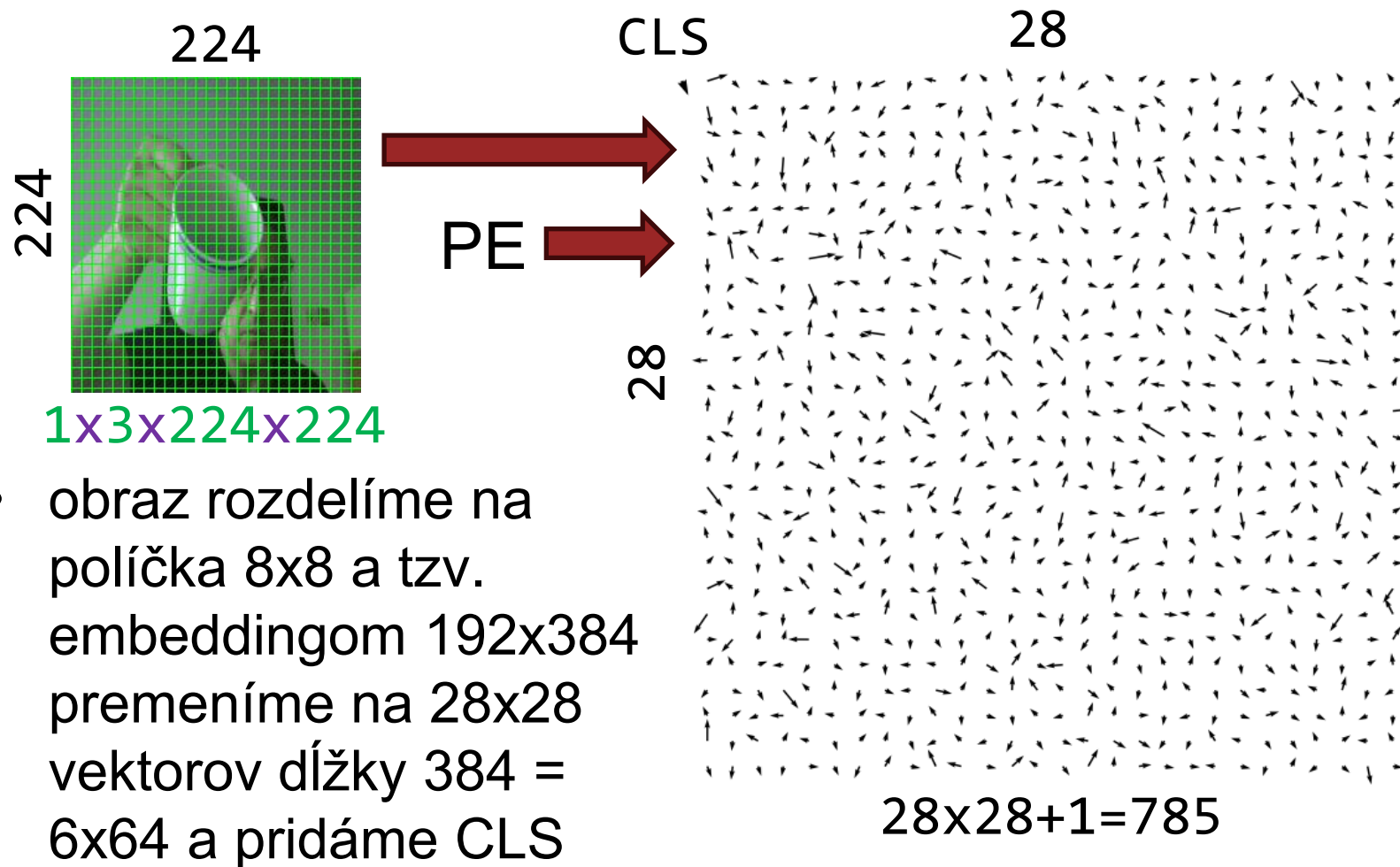


- Na spracovanie obrazu používame model DINO, ktorý obraz premieňa na 6 poznostných máp
- Pre blízke objekty najlepšie funguje mapa č. 2, takže jej ťažisko určuje zhruba pozíciu objektu
- logika otáčania hlavy je jednoduchá, akurát rozdielna pre robota a simulátor

Ako funguje DINO?

self-Distillation
with NO labels

je to špeciálne trénovaný vizuálny transformer

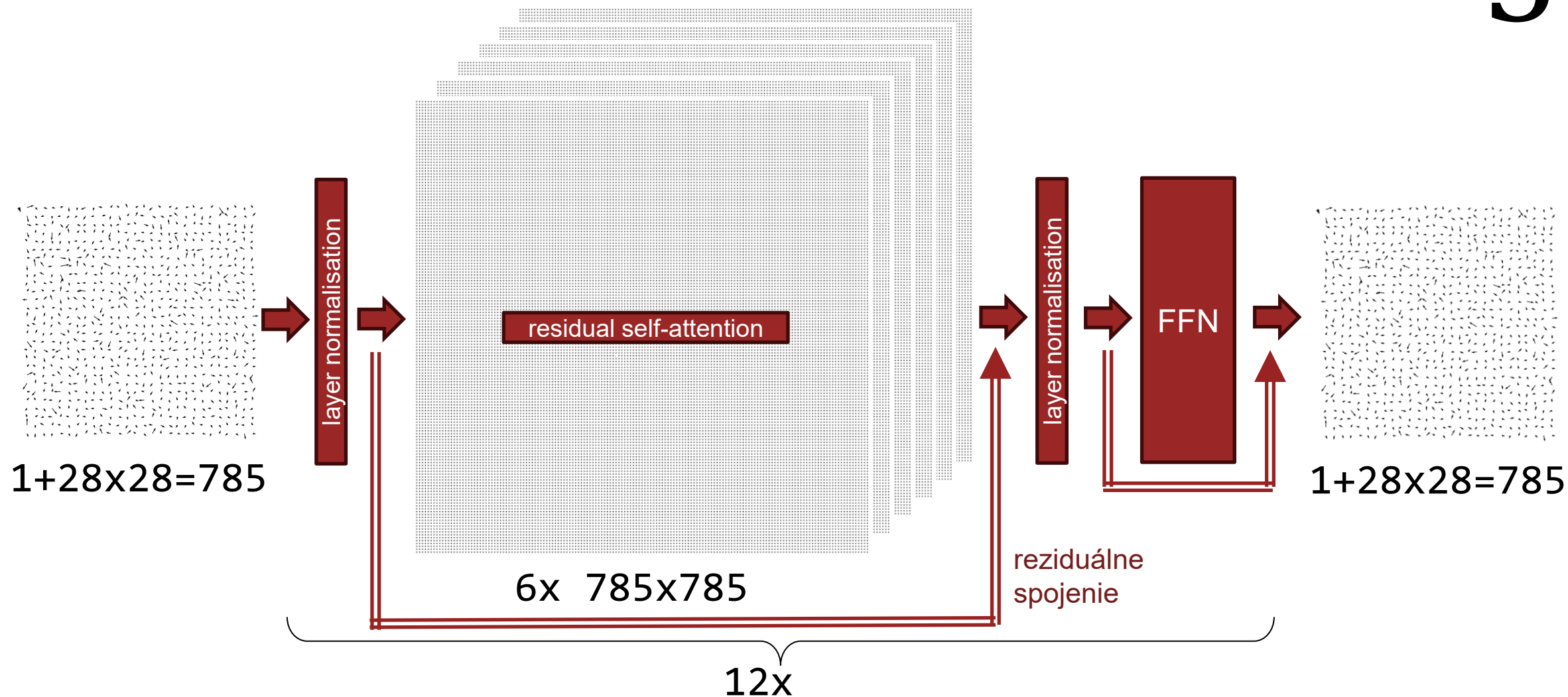


5

- tieto vektory ukazujú zhruba k povrchu 384 rozmernej hypergule a čím podobnejšie políčka reprezentujú, tým menší uhol zvierajú. Každý z 384 príznakov, predstavuje jeden spôsob, akým sa môžu dve políčka líšiť. Delia sa na 6 častí po 64.

Ako funguje DINO?

5



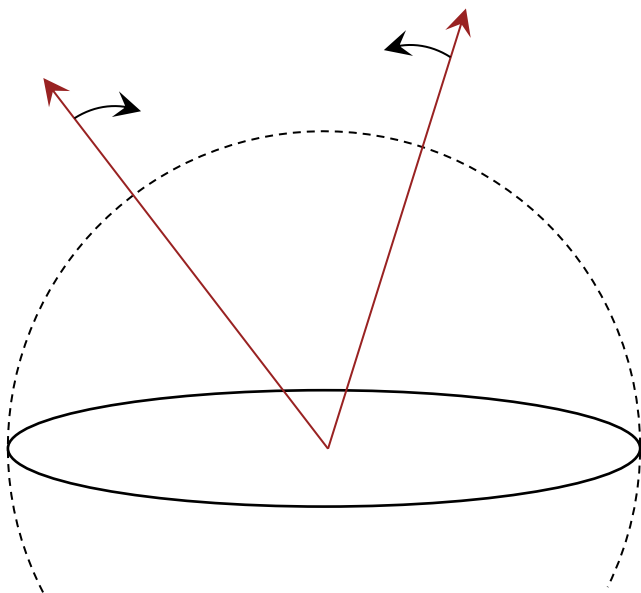
Ako funguje DINO?

residual self-attention

5

Na políčku 10,10
vidíme špicaté
ucho

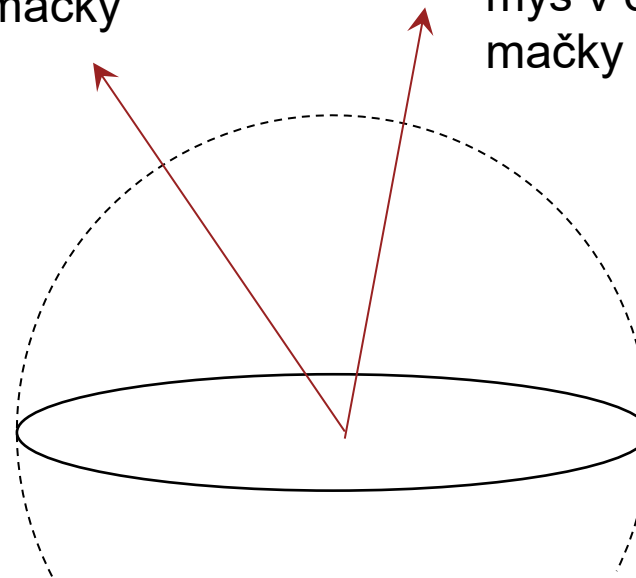
Na políčku 12,13
vidíme myš v
drápoch



*64 rozmerný priestor
na jednej zo 6 hláv*

Na políčku 10,10
vidíme špicaté
ucho mačky

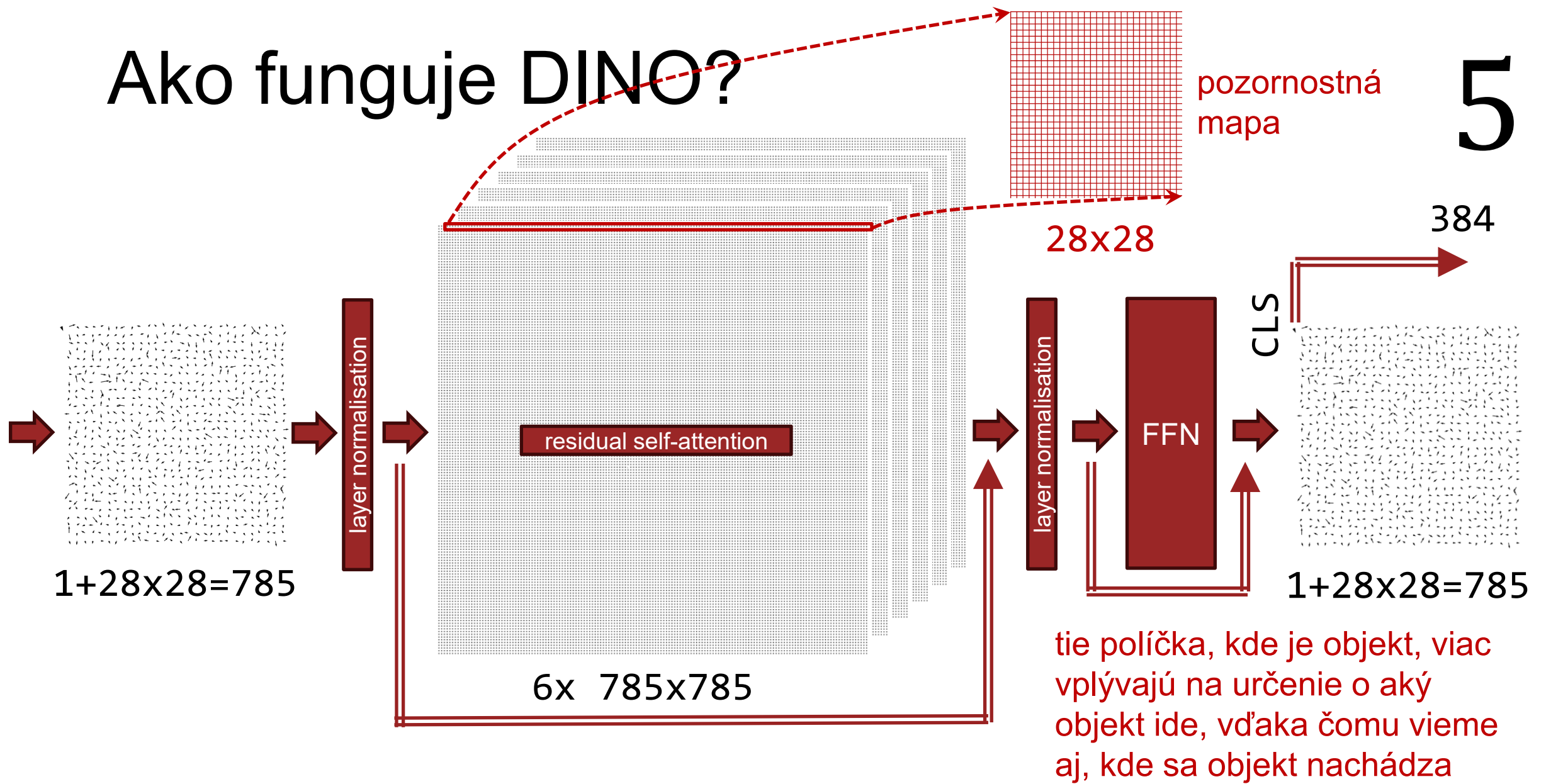
Na políčku 12,13
vidíme ulovenú
myš v drápoch
mačky



- *počítame ako sa má každý vektor posunúť v kontexte ostatných*
- *tým získavajú globálnejší význam*

Ako funguje DINO?

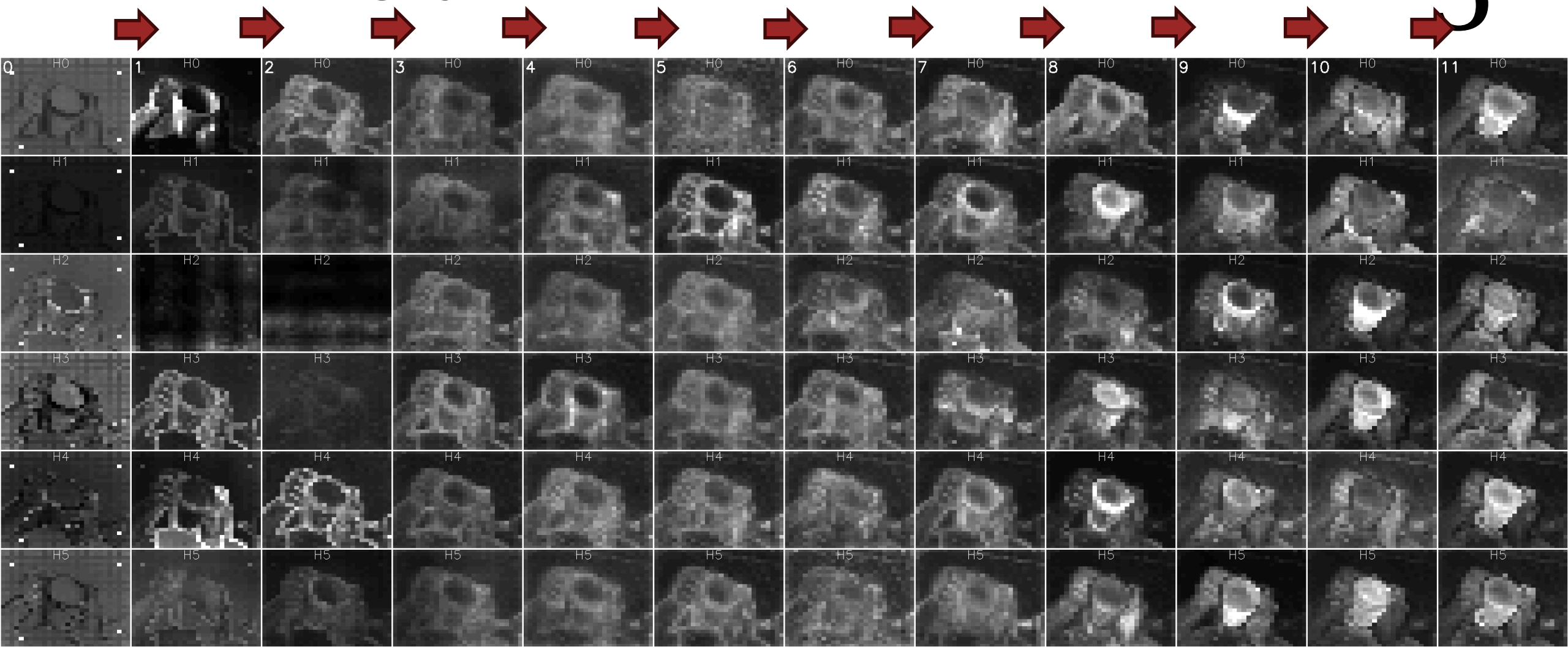
5



Ako funguje DINO?

pozornostné mapy

5



TERAIS

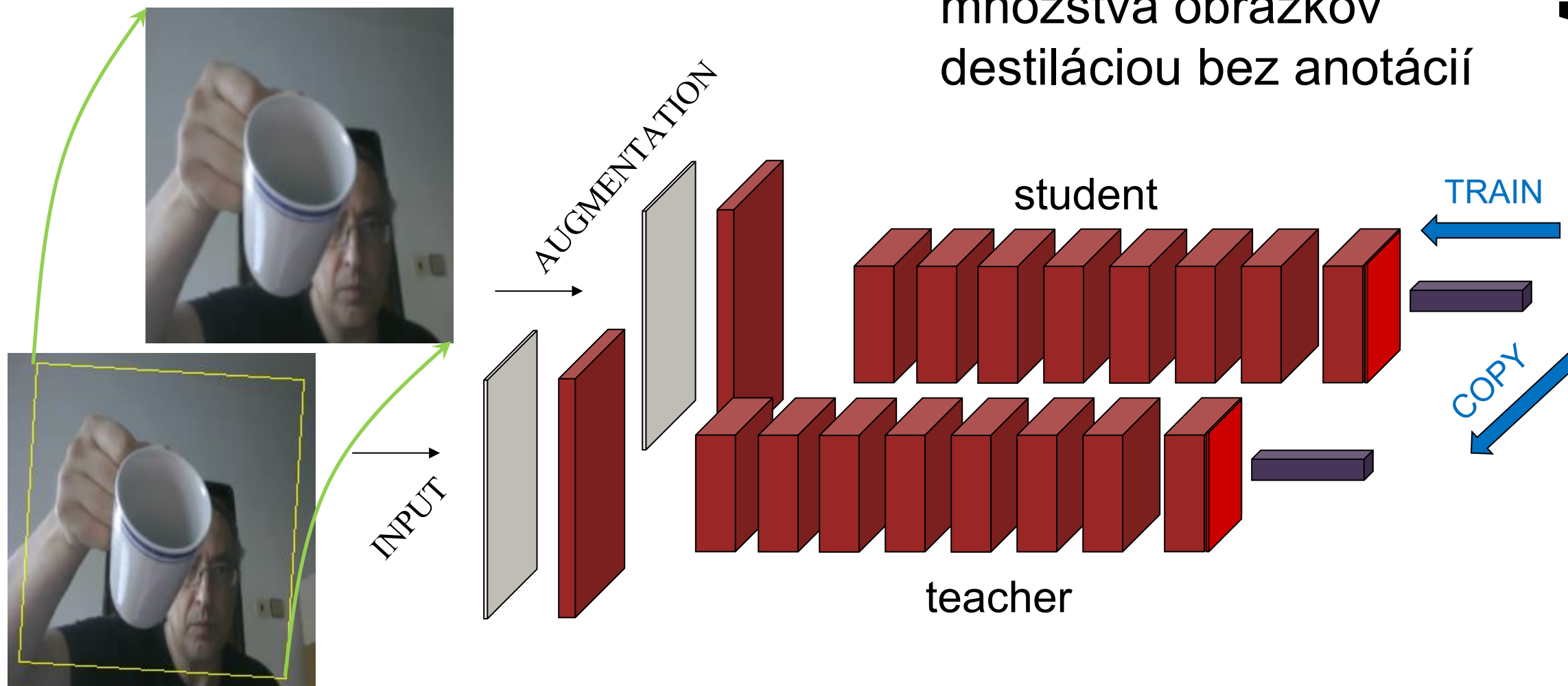


Co-funded by
the European Union

Ako funguje DINO?

- trénujeme z veľkého množstva obrázkov destiláciou bez anotácií

5





5

(Na videu farebný bod reprezentuje ťažisko pozornostnej mapy s posledného bloku transformeru. Číslo bodu zodpovedá poradiu hlavy. Vidíme, že hlava č. 2 je zhodou okolností pre náš účel najlepšia.)

Aké moduly potrebujeme?

6

Robot vie overiť či videný obraz zodpovedá položenej otázke „je toto kačička?“ (a podobne), takže potrebujeme:

-
-

Aké moduly potrebujeme?

Robot vie overiť či videný obraz zodpovedá položenej otázke „je toto kačička?“ (a podobne), takže potrebujeme:

- porovnať či tento názov zodpovedá videnému obrazu



TERAIS



Co-funded by
the European Union

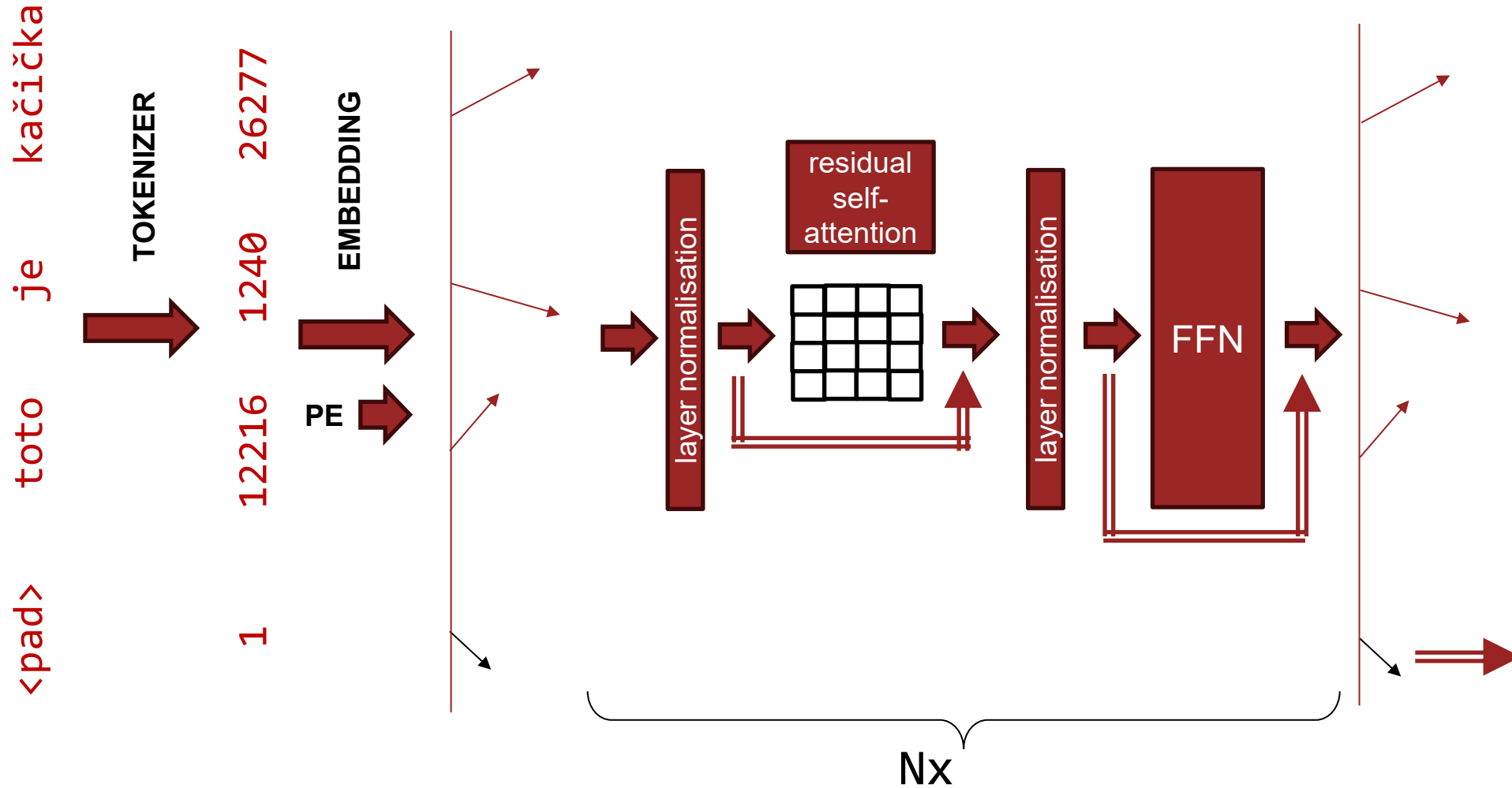
Čo potrebujeme na implementáciu?

6

- používame model CLIP
- CLIP pozostáva z dvoch encodérov: textového a obrazového
- textový premieňa text na príznakový vektor dĺžky 512
- obrazový premieňa obraz na príznakový vektor dĺžky 512
- vektory sú spriahnuté tak, že čím viac popisuje text obraz, tým menší uhol zvierajú (tým väčší je ich skalárny súčin) → kosínusová podobnosť
- dopredu si pripravíme sadu obrazových vektorov zodpovedajúcich našim konkrétnym objektom
- s nimi porovnáme vektory aktuálneho obrazu a textu
- pokiaľ zodpovedajú tomu istému objektu, kýveme a hovoríme „Áno“, inak „Nie“

Ako funguje textový transformer?

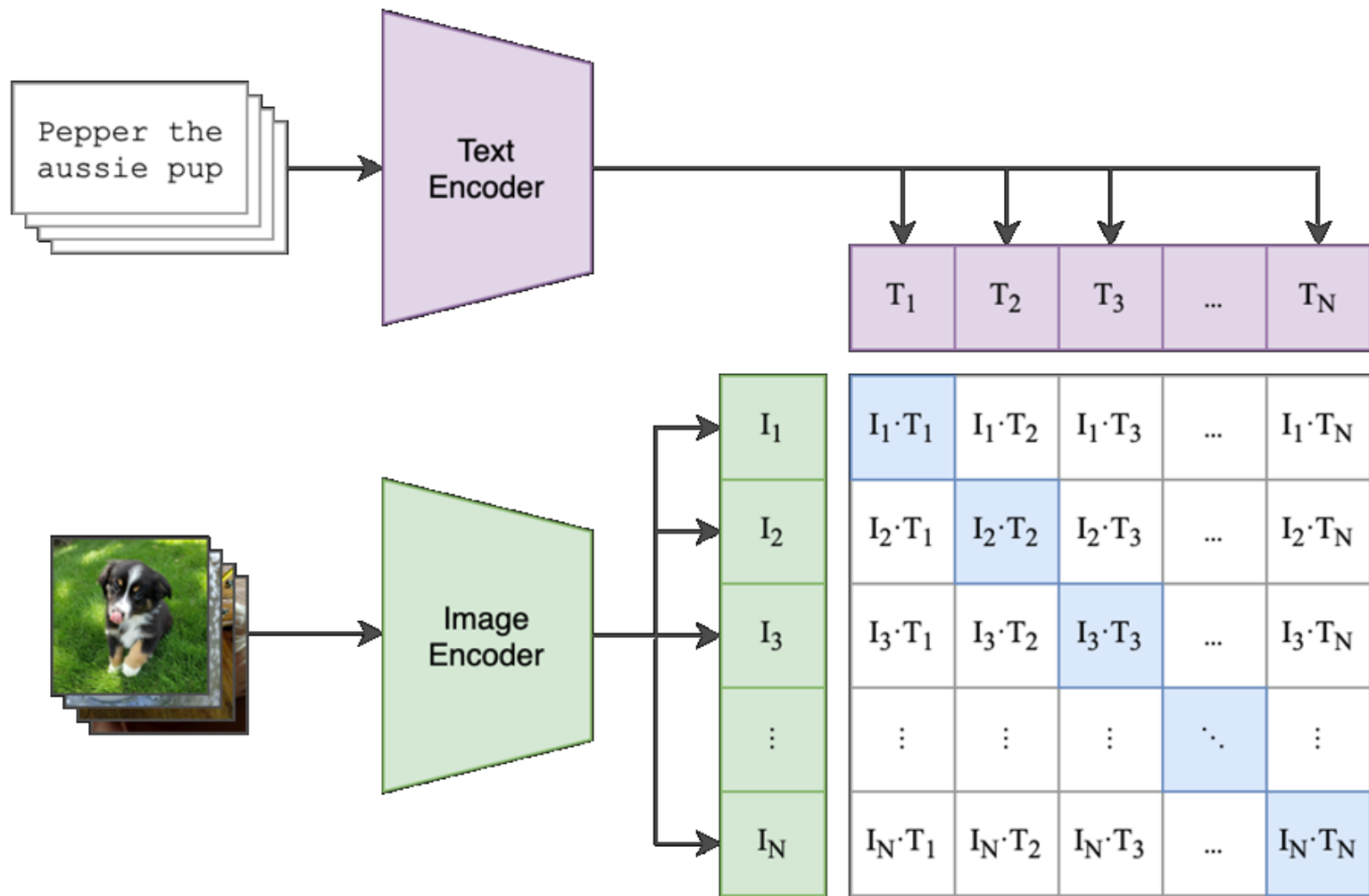
6



Ako funguje CLIP?

Contrastive Language-
Image Pre-training

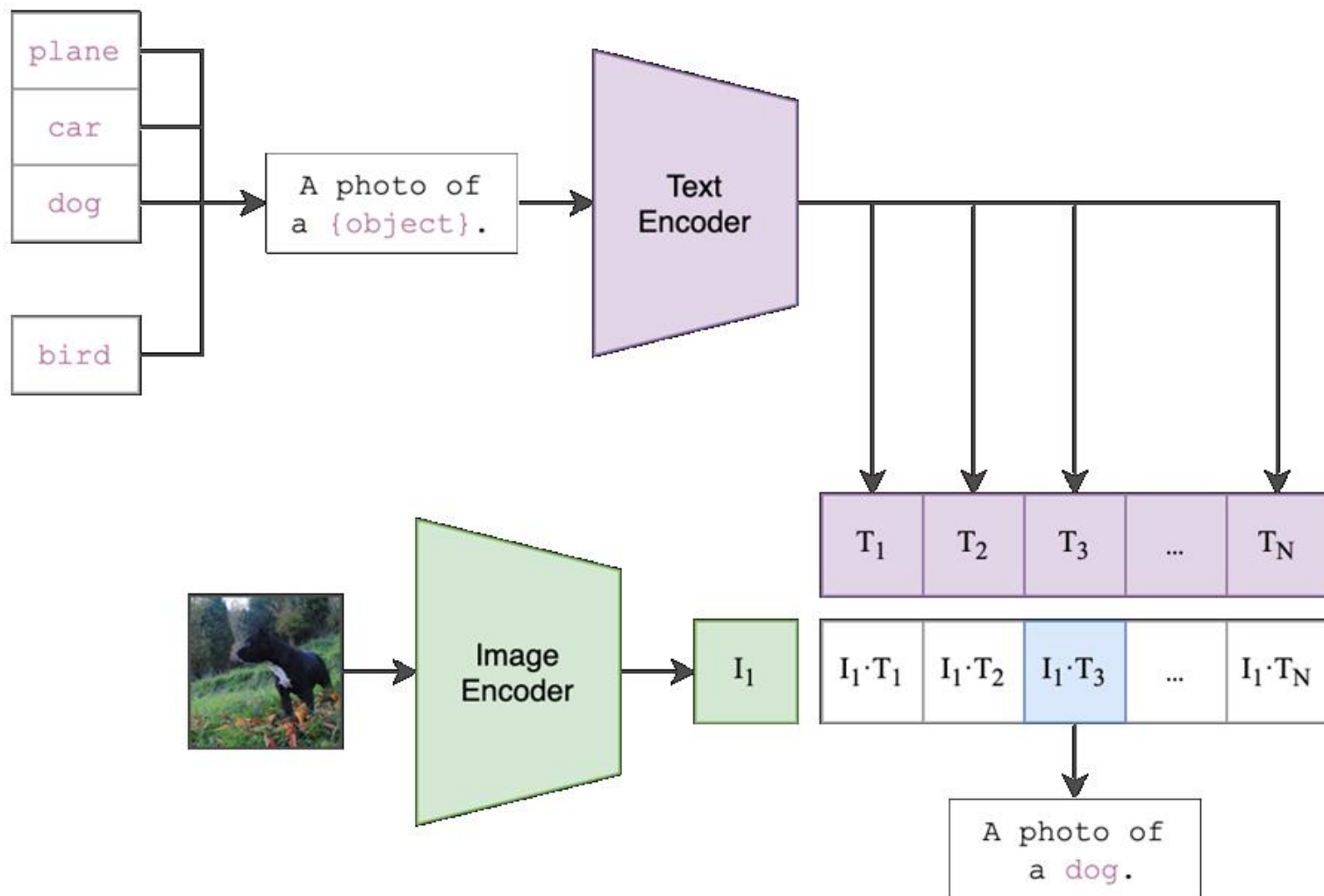
6



- CLIP je špeciálne trénovaný vizuálny transformer + textový transformer (veľký jazykový model), oba encoder-only
- ich spriahnutie sa dosahuje trénovaním s kontrastívnou chybovou funkciou, pri ktorom žiadame, aby bol skalárny súčin kódov zodpovedajúcich si vzoriek väčší než kódov tých, čo nezodpovedajú

Ako funguje CLIP?

6



- Pri používaní potom porovnávame jeden aktuálny kód (zodpovedajúci položenej otázke alebo videnému obrazu) sady dopredu pripravených kódov (z textov alebo obrazov)
- Kde najväčší skalárny súčin, tak to je pár, ktorý si zodpovedá (musí tam taký byť)



6

(Na videu ľavé červené číslo predstavuje kategóriu, pravé pravdepodobnosť. Zelené číslo v ľavom hornom rohu je počet zavolaní CLIPu za sekundu.)

Aké moduly potrebujeme?

7

Robot hovorí, takže potrebujeme:

-
-



TERAIS



Co-funded by
the European Union

Aké moduly potrebujeme?

Robot hovorí, takže potrebujeme:

- reproduktor
- syntézu reči



TERAIS

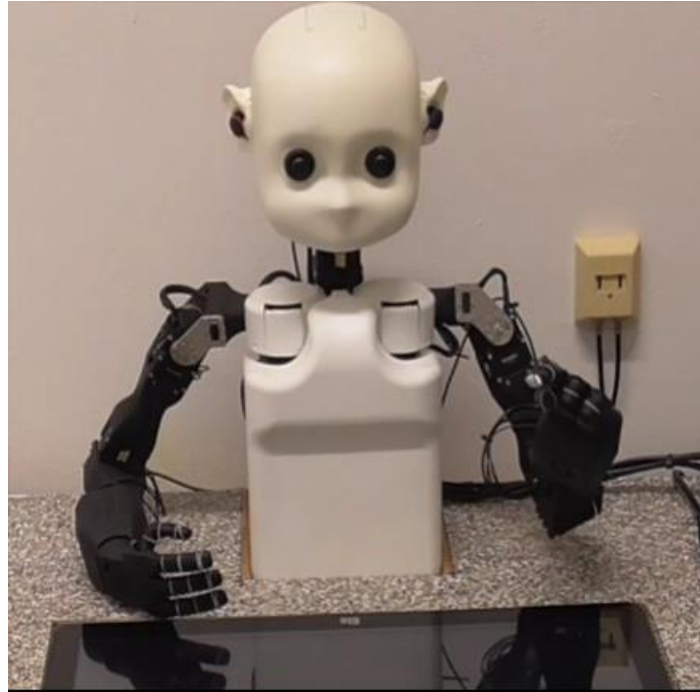


Co-funded by
the European Union

Čo potrebujeme na implementáciu?

7

```
 speak('Áno')  
 time.sleep(1)  
 speak('Nie')
```



- Na syntézu reči používame knižnicu pytttsx3 a hlas Slovak Filip (Windows) alebo espeak (Linux)

Aké moduly potrebujeme?

8

Robot počuje, takže potrebujeme:

-
-

Aké moduly potrebujeme?

Robot počuje, takže potrebujeme:

- mikrofón
- rozpoznávanie reči



TERAIS



Co-funded by
the European Union

Čo potrebujeme na implementáciu?

8

- Je dobré, ak má mikrofón Push-To-Talk
- používame model Whisper
- Robot nesmie počúvať, keď sám hovorí

Aké moduly potrebujeme?

Robot vie z počutého textu rozpoznať o aký predmete je reč:

-
-



TERAIS



Co-funded by
the European Union

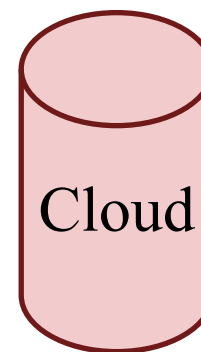
Čo potrebujeme na implementáciu?

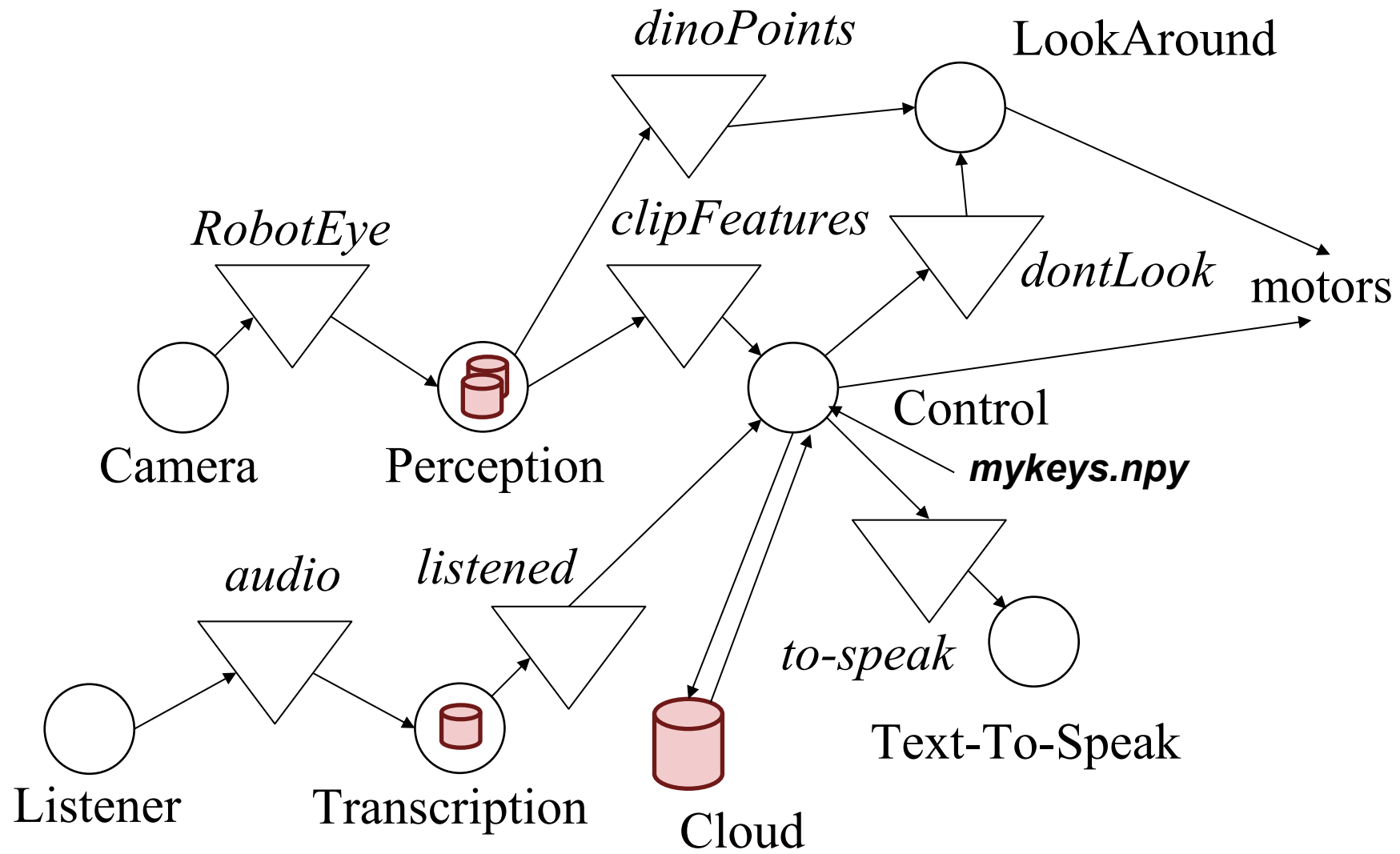
9

- Regex
- Je toto kačička?
Je toto (.*)?

alebo

- LLM
- Prompt
Examples
Toto je tiež psíček?
-
- ← pes





Ďakujem za pozornosť!

lucny@fmph.uniba.sk

<https://github.com/andyLucny/OAI/workshop>

<https://github.com/andyLucny/Recognize4PC>

<https://github.com/andyLucny/nico>

<https://github.com/andyLucny/nico2>

<https://github.com/andyLucny/whatisthis>

<https://github.com/andyLucny/drawthispicture>



TERAIS



Co-funded by
the European Union